



Map Reduce

Michael Schöttner

- Motivation
- MapReduce Programming model
- MapReduce System architecture
- Example: PageRank
- Apache Hadoop

- MapReduce was introduced by Google in 2004
- Big Data @Google:
 - 20+ billion web pages x 20 KB ~ 400+ TB
- One computer can read 30-35 MB/sec from disk
 - Need 4 months to read the web from one disk
 - 1.000 hard drives just to store the web
- Even more time/HDD, to do something with the data:
 - Data processing
 - Data analytics

- Good news: “easy” parallelization of reading from disk
 - Reading the web with 1000 machines $\Rightarrow$ less than 3 hours
- Bad news: programming work for parallel analytics
 - Communication and coordination
 - Debugging
 - Fault tolerance
 - Management and monitoring
 - Optimization
- Worse news: repeat this for every problem

What is needed?

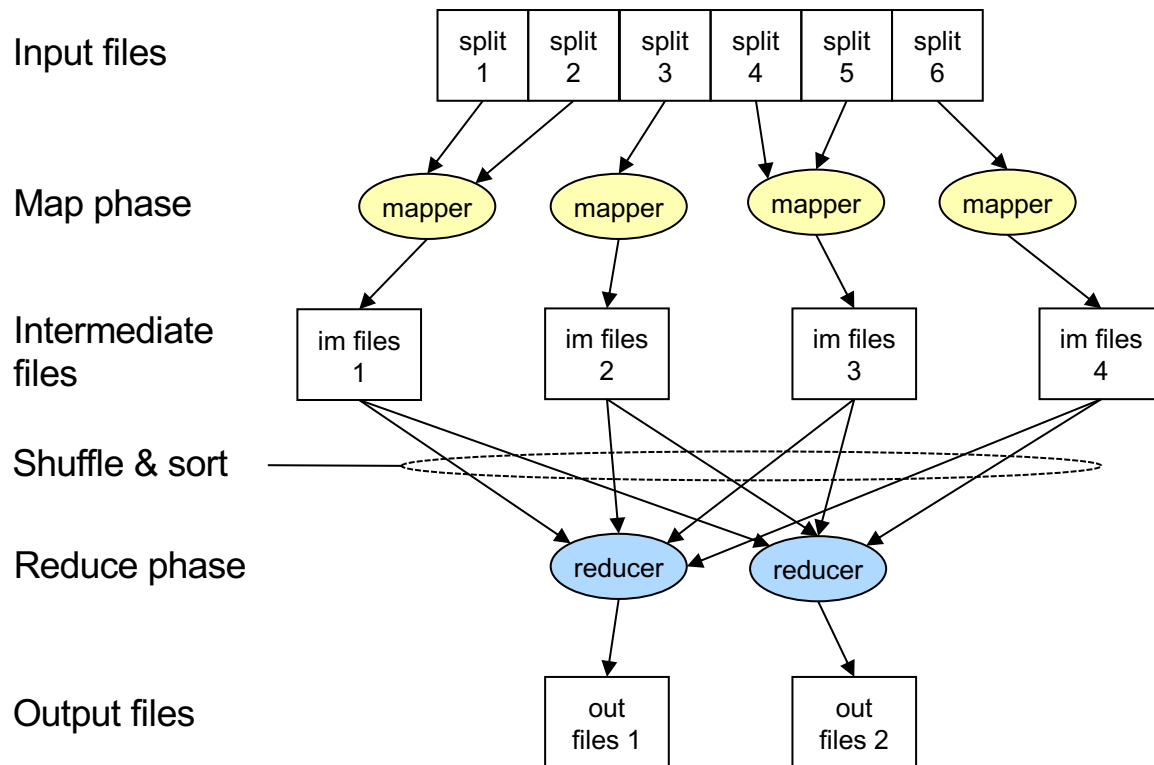
- A simple programming model that applies to many data-intensive computing problems
- Main objective: hide complex details of parallelization
 - Task management and load balancing
 - Data partitioning
 - Network and disk transfer
 - Handling of failures



Programming model

- Programmer has to implement only two functions: **map** and **reduce**
 - Inspired by Lisp (functional programming language), developed in 1958
- Typical steps by MapReduce
 - Read a lot of data
 - **Map**: extract something interesting from each record
 - **Reduce**: aggregate, summarize, filter or transform
 - Write the results

MapReduce at a glance



- Map outputs intermediate results as tuples (key, value)
- Example: word counting in a text file

```
map(filename, value) {  
    for each word w in value  
        emit(w, "1");  
}
```

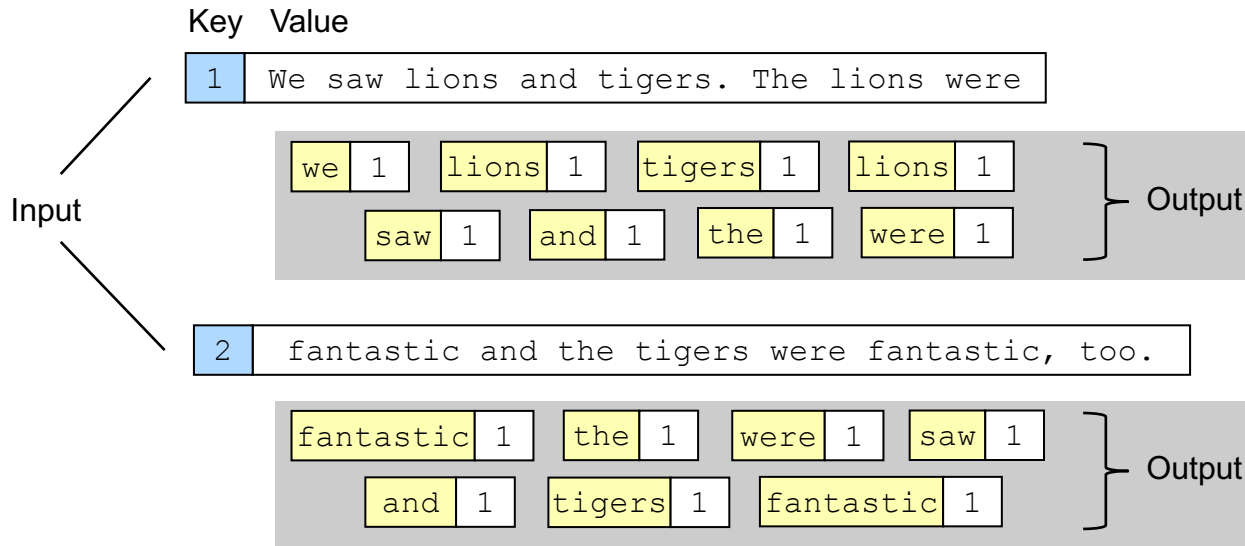
Content of the file

For each word emit a tuple (word, 1). If the same word occurs several times in `value`, several tuples are emitted.

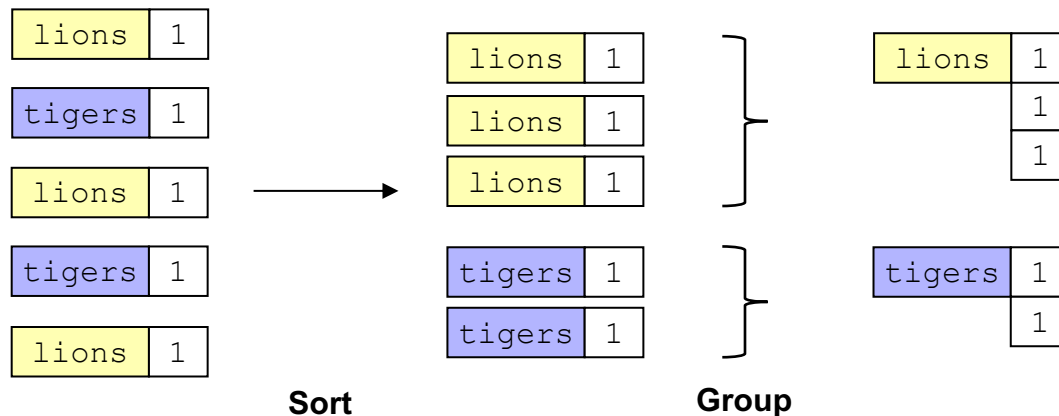
- Remark:
 - Input (filename, value) is also a tuple but different to the emitted tuples

■ Example: word counting in a text file

- Input: one text row (key=row number, value=content of the text row)
- Output: tuples (key=word, value=1)



- **Optional** operation; executed on each mapper
- Includes local sorting and grouping by key (done in memory)
 - Saves storage and reduces network bandwidth requirements
 - Only for commutative and associative functions, e.g. SUM, MAX, MIN, COUNT, ...



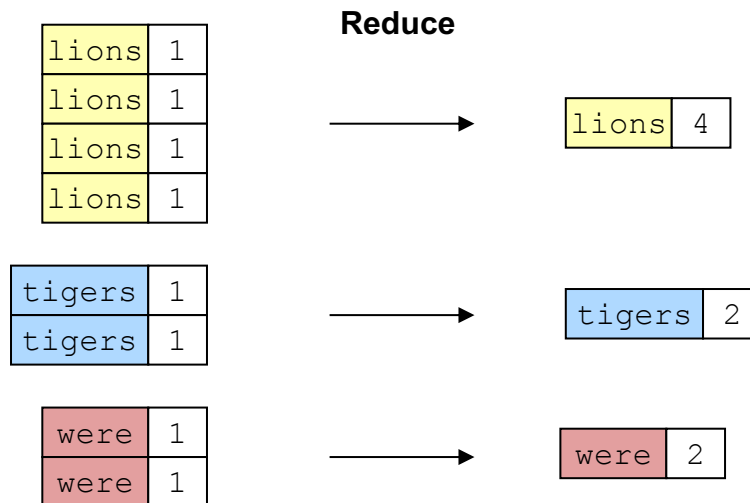
- Reduce combines the intermediate results from mappers
- Example continued

```
reduce(word, value) {  
    int result = 0;  
  
    for each w in value  
        result += w;  
  
    emit(word, result);  
}
```

List of word frequencies.
Tuples are grouped using same keys. The list is accessed using an iterator (allowing processing large lists.

Emit sum of word frequencies.

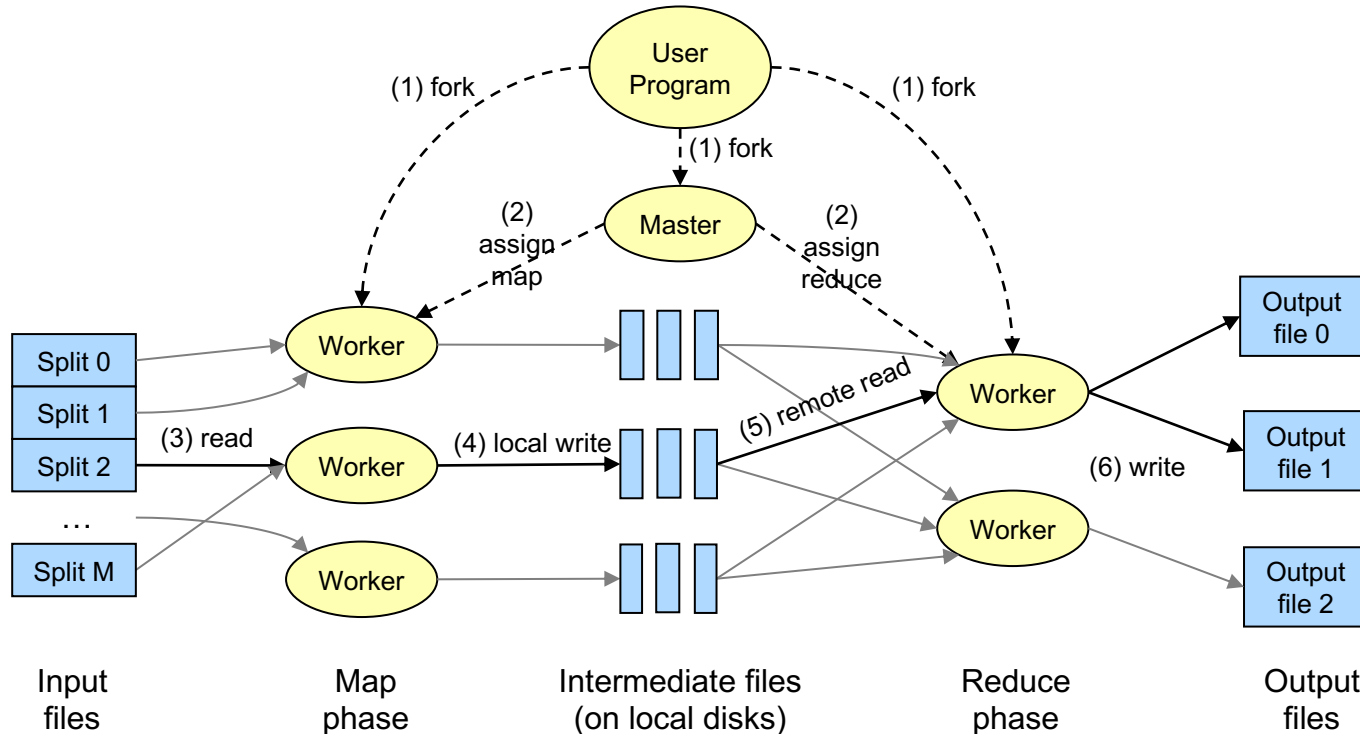
- Example: word counting in a text file
 - Input: sorted lists of tuples
 - Output: list of tuples (typically aggregating input)
 - Reduce can run in parallel



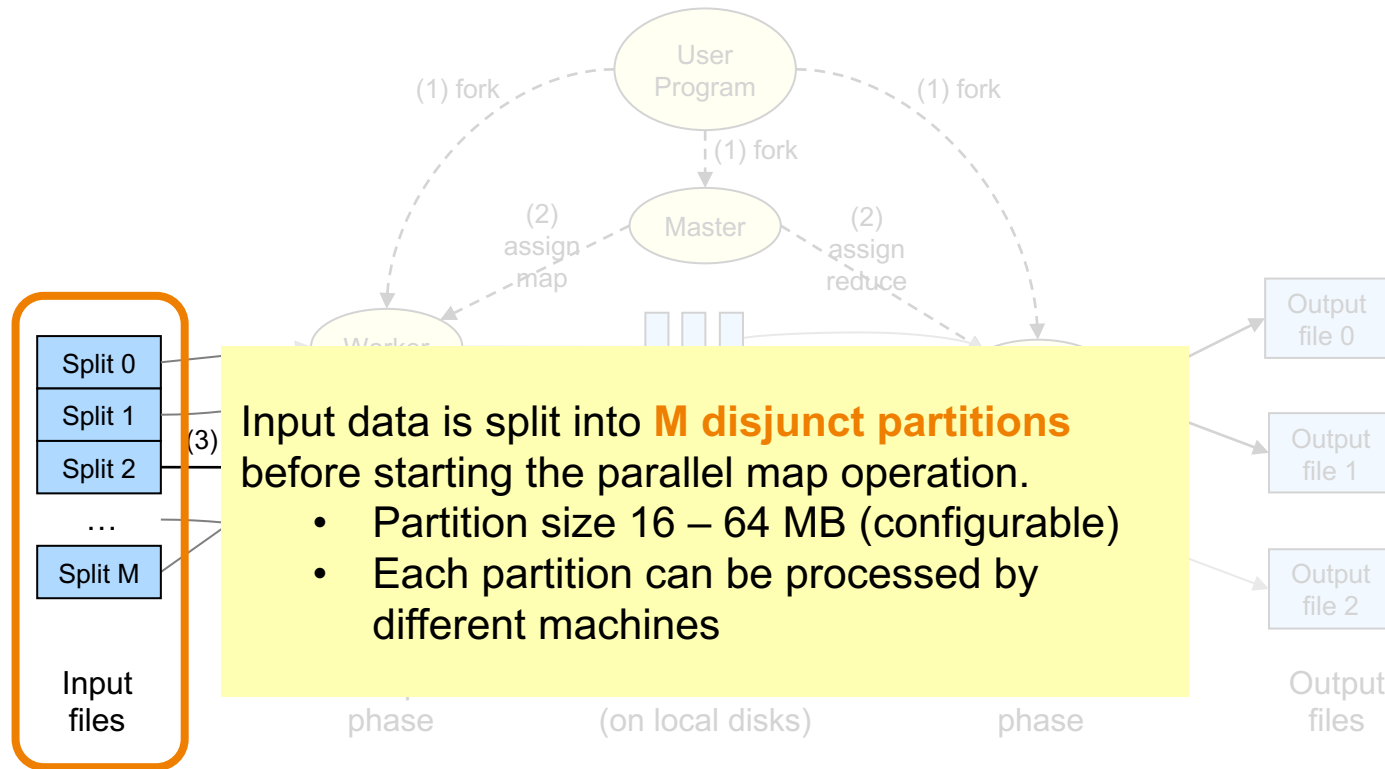


System architecture

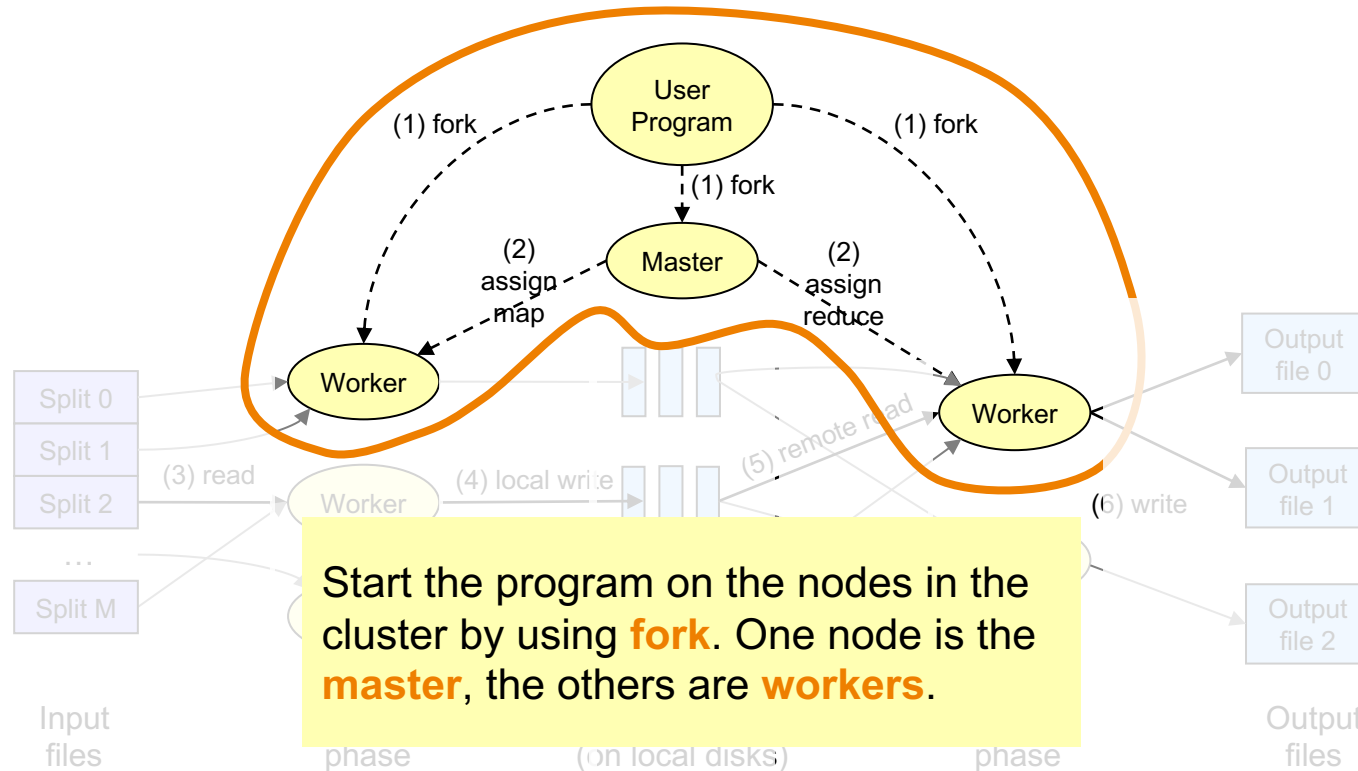
MapReduce System Architecture



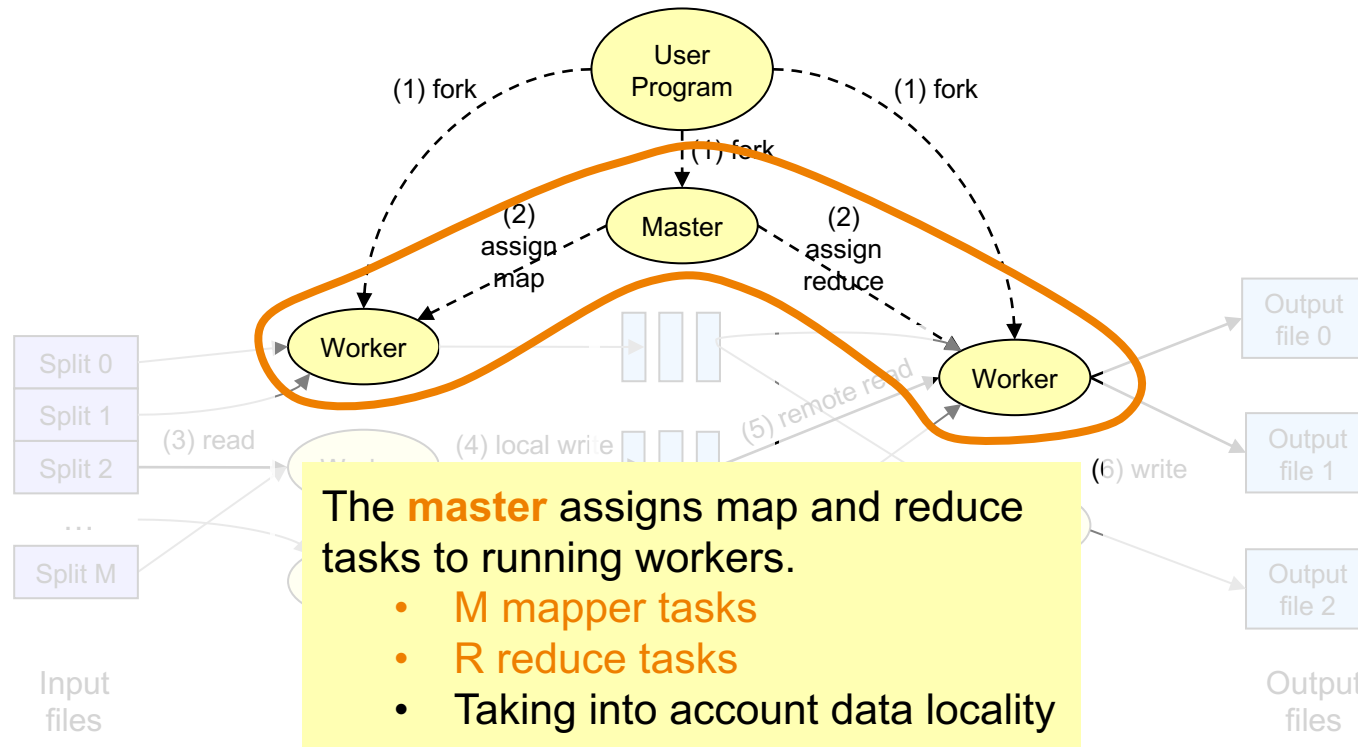
(1) Splitting input data



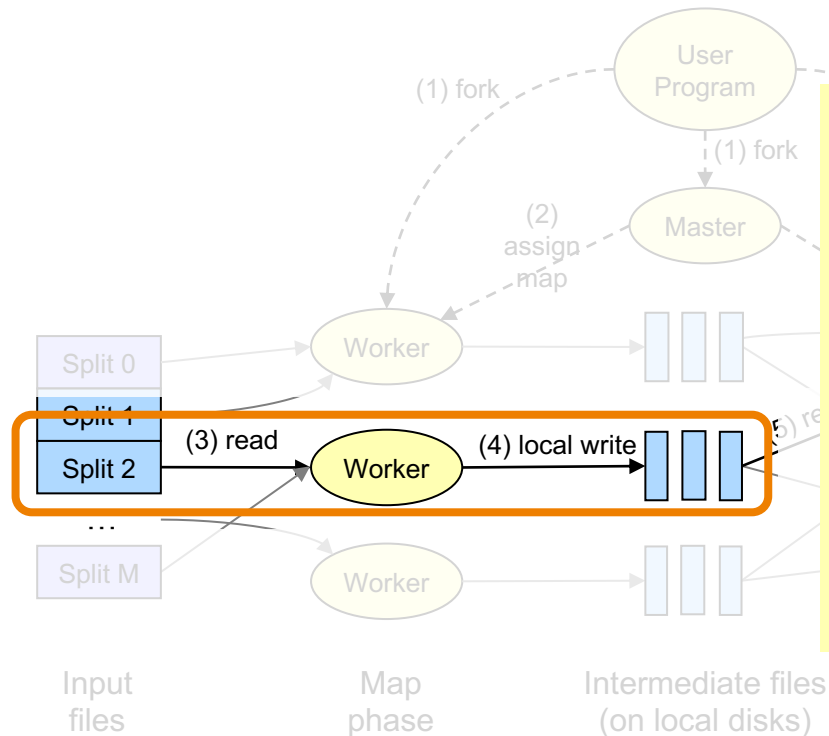
(2) Starting program instances



(3) Assigning tasks



(4) Running map task



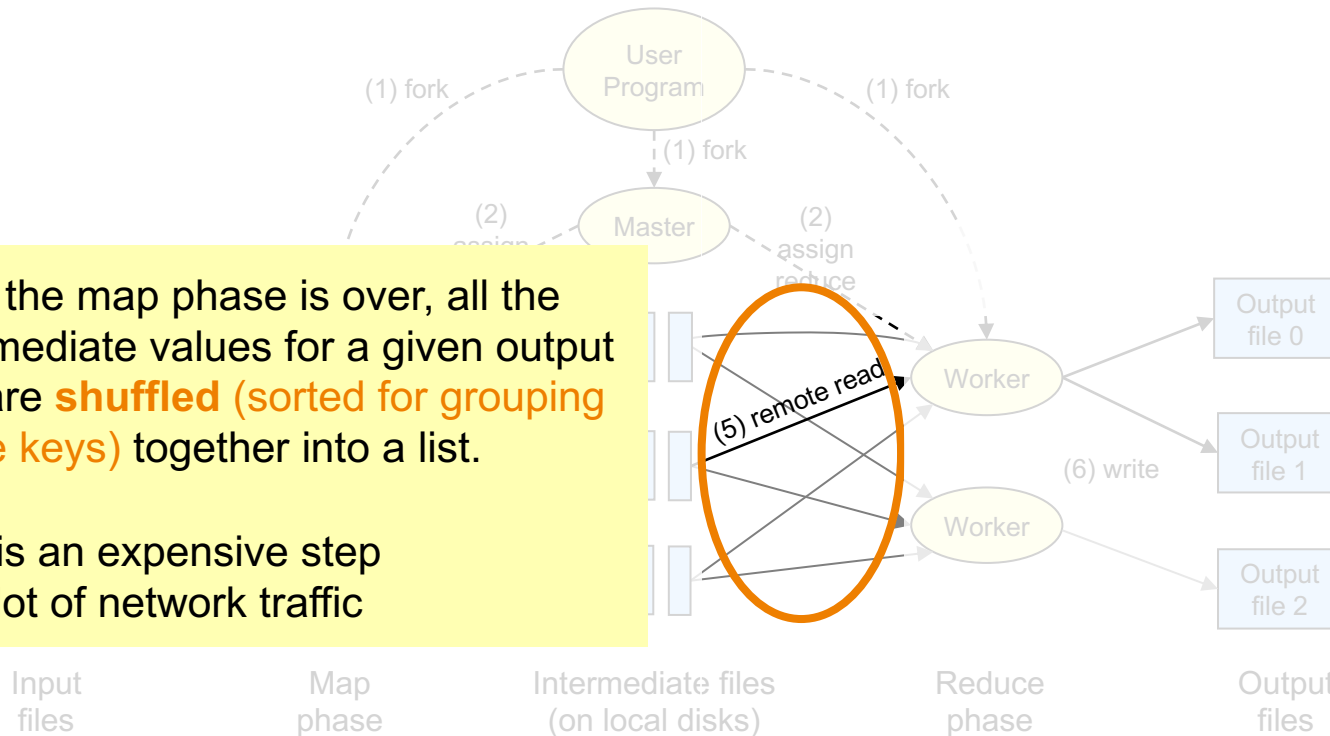
Worker running map task:

- Reads data from assigned split (3)
- Processes all **input tuples** with its map function
- Results are written to local disk (4)
 - The output is partitioned in **R files / partitions** (e.g. $\text{hash}(\text{key}) \bmod R$)
 - Partition/file locations are reported to the master (needed for later assigning reduce tasks)

(5) Shuffle phase

After the map phase is over, all the intermediate values for a given output key are **shuffled** (sorted for grouping same keys) together into a list.

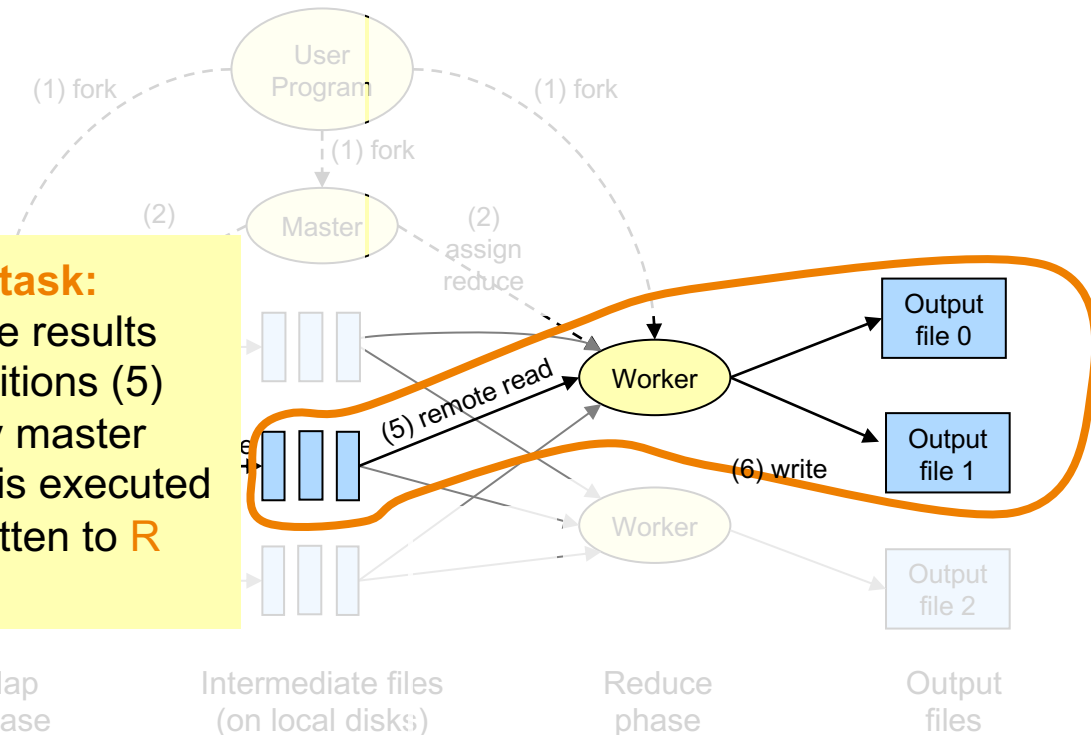
This is an expensive step
→ a lot of network traffic



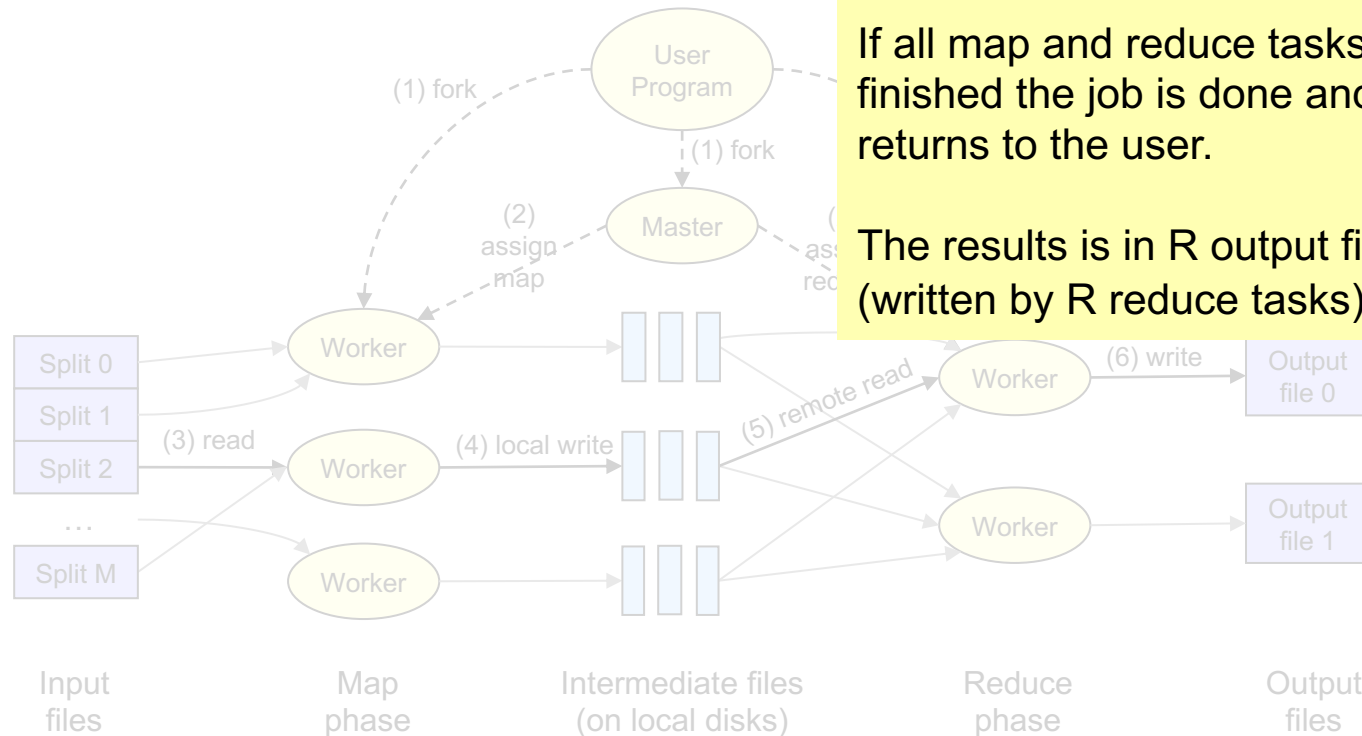
(6) Running reduce task

Worker running reduce task:

- Reads intermediate results from mappers partitions (5)
- Location is sent by master
- **Reduce function** is executed and results are written to **R output files**.



(7) Termination



- **Map** functions run in parallel, creating different intermediate values from input splits
- **Reduce** functions also run in parallel, each working on a different output key
- Important: all values are processed independently!
- Bottleneck: reduce phase cannot start until map phase is completely finished
→ all reducers have to wait for the slowest map task
- Typical configuration: 1 split / chunk = 64 MB, M=200.000; R=4.000; workers=2.000
 - M and R should be larger than the number of workers to allow load balancing

■ Worker

- Master monitors workers by using a heart-beat protocol
- After a configurable timeout tasks of a failed worker are re-assigned to another alive worker
 - Includes all tasks of a failed worker, also completed ones
- Single tasks can be restarted because they are independent from each other
- Re-computing data is better than checkpointing & restart workers

■ Master

- Checkpointing state in a distributed file system
- Allows restarting the master in case of crash on the same or a different server
- Backup master can run in cold or hot standby

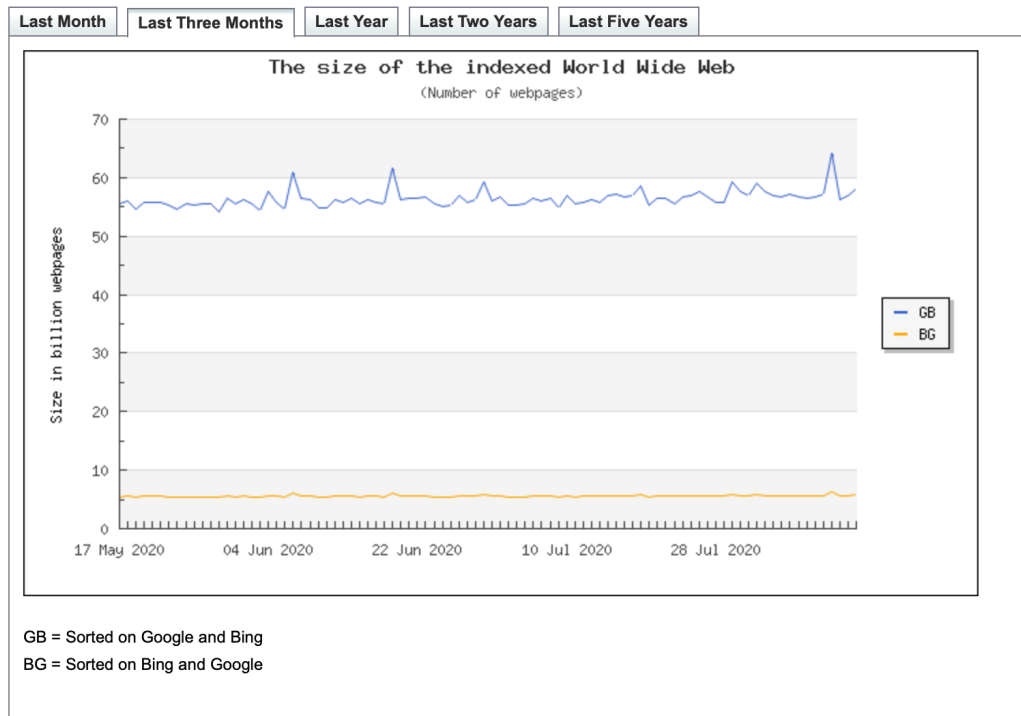
- Not built for real-time processing
- Very limited queries:
 - Difficult to write more complex tasks
 - Solution: declarative query languages, e.g. Hive
- No built-in support for iterative processing
- Barrier between Map and Reduce
- Shuffle-Phase is very expensive because of a lot of network traffic

- Input and output data is stored in GFS or Bigtable
- MapReduce: Simplified Data Processing on Large Clusters,
Jeffrey Dean and Sanjay Ghemawat, 2004
<https://research.google/pubs/pub62/>



Example: PageRank

- PageRank (PR) is an algorithm used by Google Search to rank web pages in their search engine results.
 - PR was named after Larry Page, one of the founders of Google.
- Input data is the web
 - ~55 billion linked web pages
 - Each ~2 MB
 - In total around **>1 PB**



<https://www.worldwidewebsize.com>

- WWW around Wikipedia →



- Input: world wide web (linked documents)
- Output: ranking of each web page depending on global structure
- Basic idea: The rank of a page depends on how often it is referenced and on the rank of the referencing pages
- Inventors: Larry Page & Sergei Brin (Stanford University)
 - Founders of Google
 - Page rank is used to sort search results
 - Different variations of the algorithm exist which are not publicly available

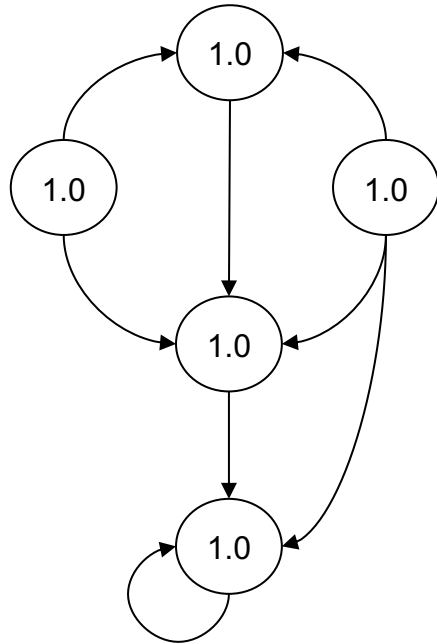
- Computing page rank (PR) of page A

$$PR(A) = (1 - d) + d * \sum_{i \in Refset} \frac{PR(i)}{C(i)}$$

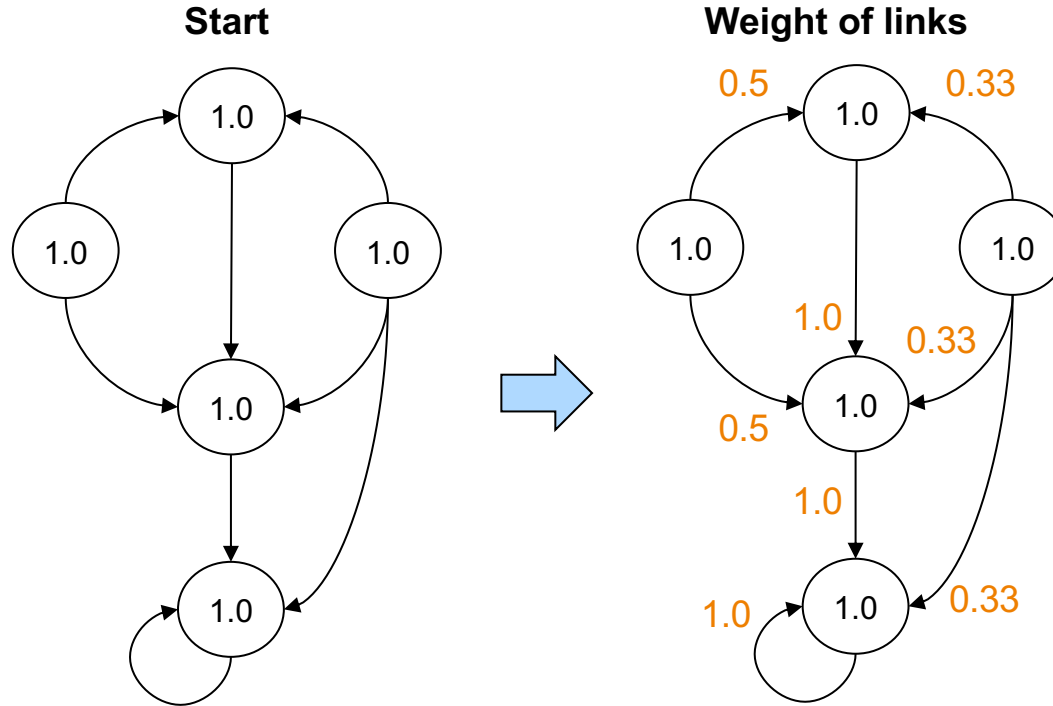
- Refset: set of pages referencing page A
- d: damping factor [0..1], d=0.85 is often used
 - d is the probability (uniform distribution) of a random surfer using a link on the current page. or he can also input a new link.
 - d=0: surfer never selects a link → all pages have the same PR
 - d=1: surfer never inputs a new link → far away pages get a very low PR and may never be listed
- PR(i): page rank of page i
- C(i): number of outgoing links on page i

PageRank Example

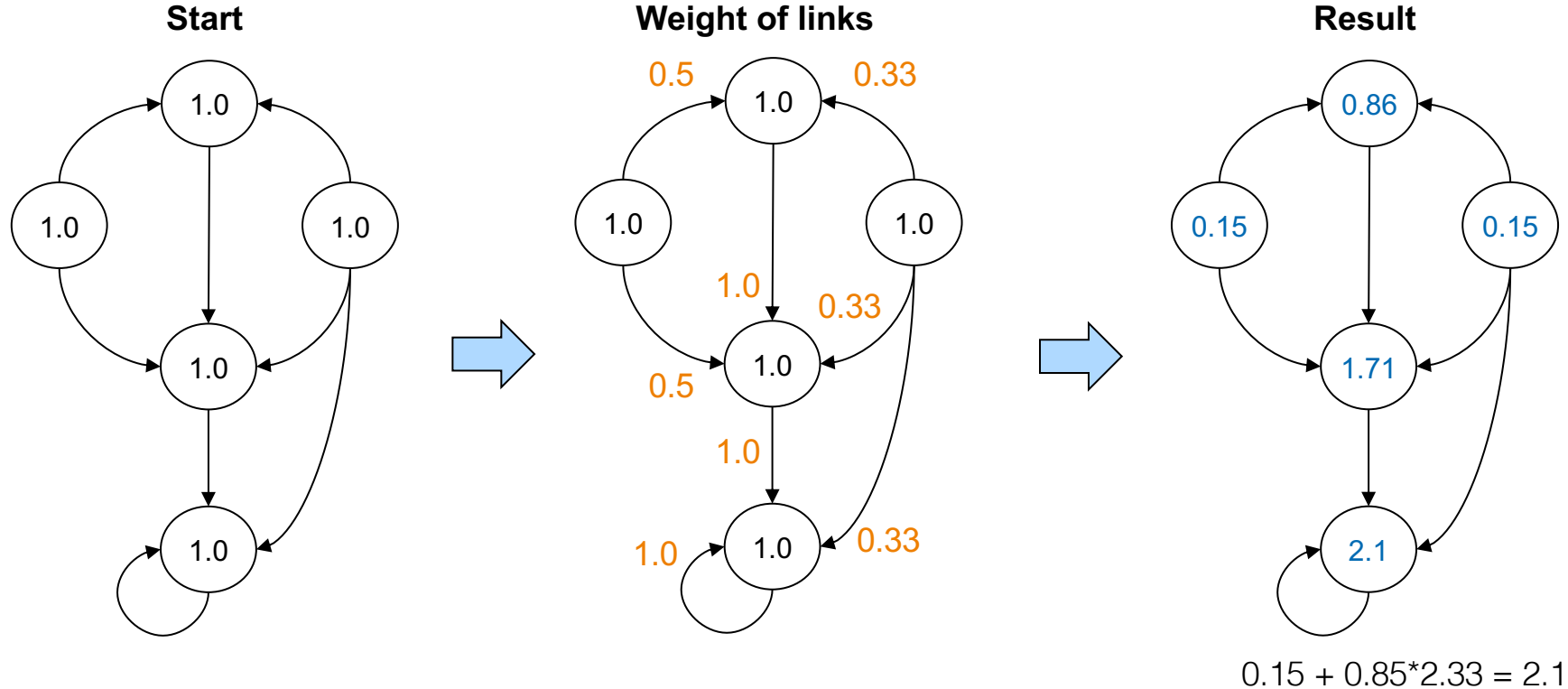
Start



PageRank Example



PageRank Example



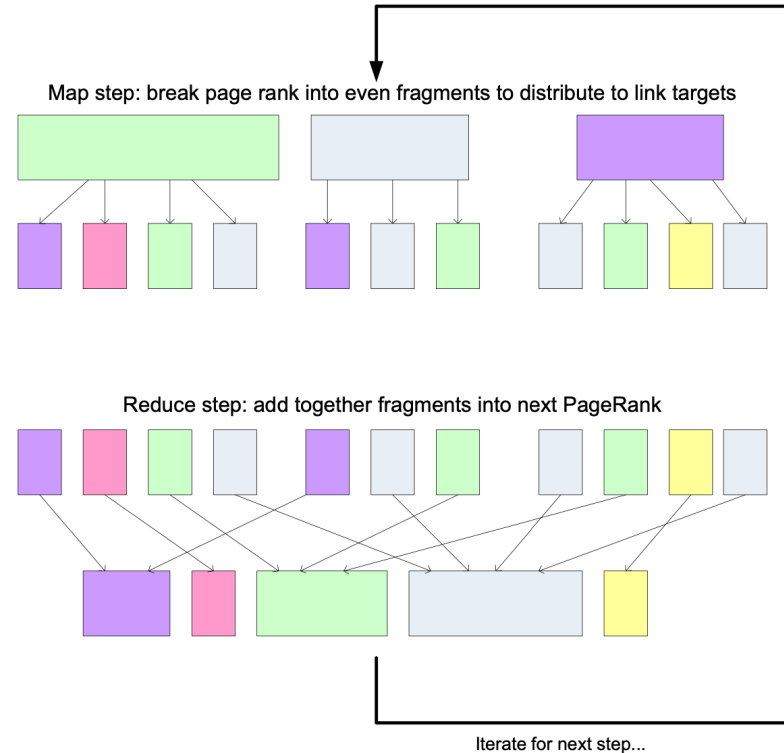
- Again we need an iterative implementation because $PR(i+1)$ is based on $PR(i)$
- Initialization:
 - Two tables: `current` and `next` each row storing $PR(i)$ for i -th page
 - Entries in `current` are initialized with 0 or 1
 - The default value for d is 0.85
- Loop:
 - Iterate over all pages, distributing PR from `current` into `next` of link targets
 - `current := next; next := fresh_table();`
 - Next loop iteration or stop (e.g. if converged)

■ Key insights:

- The `next` table depends on `current`, but not on any other rows of `next`
- Thus rows can be processed in parallel

■ Consequences of insights:

- Mapper:
We can map each row of `current` to a list of PageRank “fragments” to assign to link targets
- Reducer:
These fragments can be reduced into a single PR value for a page by summing
- Graph representation can be even more compact; since each element is simply 0 or 1, so we only transmit column numbers where it's 1



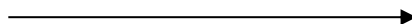
- Map task takes (URL, page content) pairs and maps them to (URL, (PR_{init} , list-of-urls))

- PR_{init} is the initial PageRank for URL
 - E.g. 0 or 1 or any value from google databases for prominent pages
 - list-of-urls contains all pages pointed to by URL

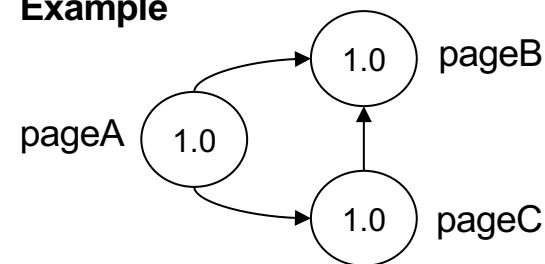
- No reduce task (in this step)

- Result of this step:

- Data stored as adjacency list
(URL, (PR_{init} , list-of-urls))



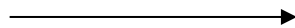
Example



Key:	Value:	
pageA	1.0	(pageB, pageC)
pageB	1.0	()
pageC	1.0	(pageB)

PR – Step 2: Map-Task

- Input: (URL, (cur_rank, list-of-urls))



Key:	Value:
pageA	1.0 (pageB, pageC)
pageB	1.0 ()
pageC	1.0 (pageB)

- Output:

- 1. $\forall u \in \text{list-of-urls}$: `emit(u, cur_rank / |list-of-urls| )`
- 2. `emit (URL, list-of-urls)`
(to carry the points-to list along through iterations)



Key:	Value:
pageB	0.5
pageC	0.5
pageA	1.0 (pageB, pageC)
pageB	1.0 ()
pageB	1.0
pageC	1.0 (pageB)

- The output of each mapper is a summand of the sum:

$$PR(A) = (1 - d) + d * \sum_{i \in \text{Refset}} \frac{PR(i)}{C(i)}$$

PR – Step 3: Reduce-Task

- Input: (URL, list-of-urls) and many (URL, val) values →
 - Sorted during shuffle

Key:	Value:
pageA	1.0 (pageB, pageC)
pageB	0.5
pageB	1.0
pageB	1.0 ()
pageC	0.5
pageC	1.0 (pageB)

- Output:
 - Sum vals and fix up with $d=0.85$
 - emit (URL, (new_rank, list-of-urls))

Key:	Value:
pageA	0.15 (pageB, pageC)
pageB	1.43 ()
pageC	0.56 (pageB)

PR – Step 4: Check if done

- Determine whether convergence has been achieved
 - E.g. check selected prominent pages and if their PR changes are beyond a given delta
- Or if a fixed number of iterations is finished

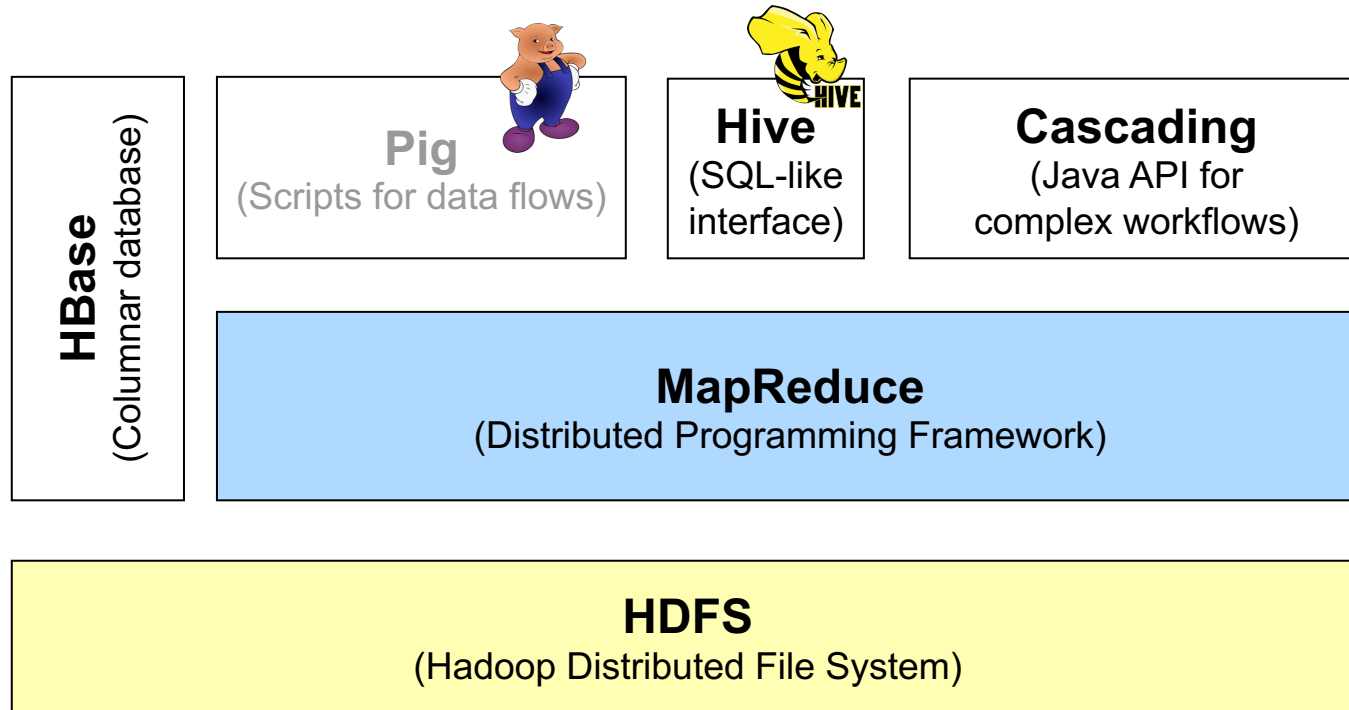


Apache Hadoop

- Hadoop is a large scale, open source software framework started at Yahoo!
 - Developed in Java
 - Dedicated to scalable, distributed, data-intensive computing
 - Handles thousands of nodes and petabytes of data
- Many sub projects
 - Including MapReduce and HDFS
- Used by many big companies, e.g. Yahoo!, Microsoft, Facebook



The Hadoop 1.0 stack



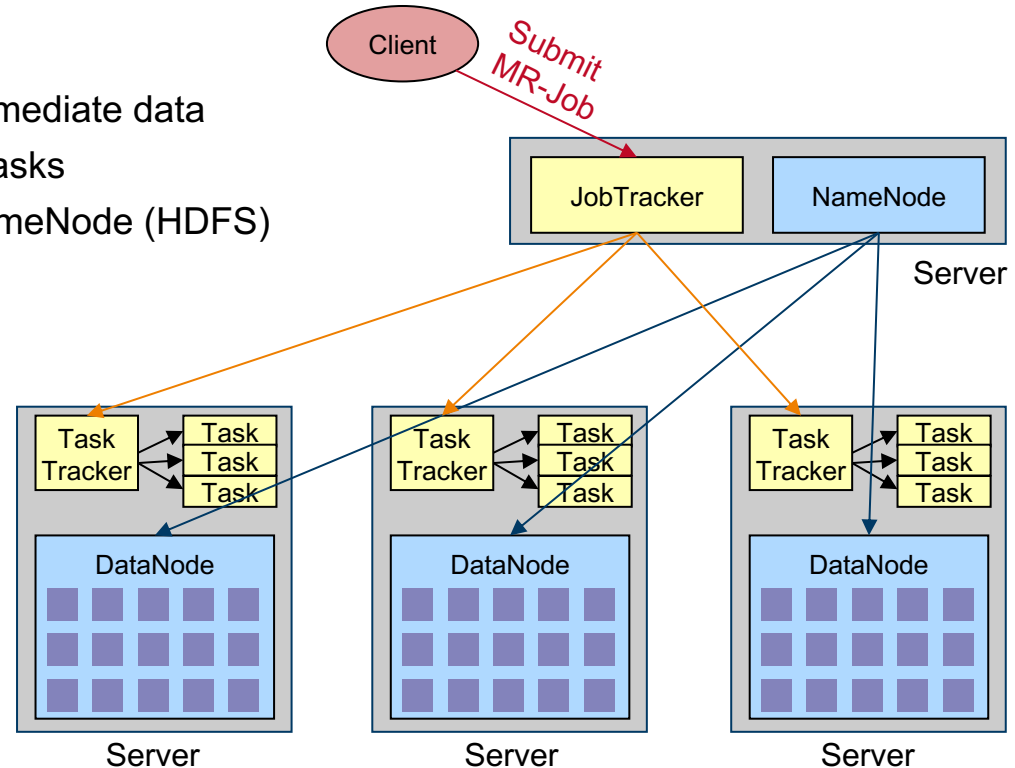
Hadoop MapReduce and HDFS

■ JobTracker:

- Location of inputs, outputs and intermediate data
- Assigns and monitors MapReduce tasks
- Co-located and coordinated with NameNode (HDFS)

■ TaskTracker

- Run tasks on the data nodes



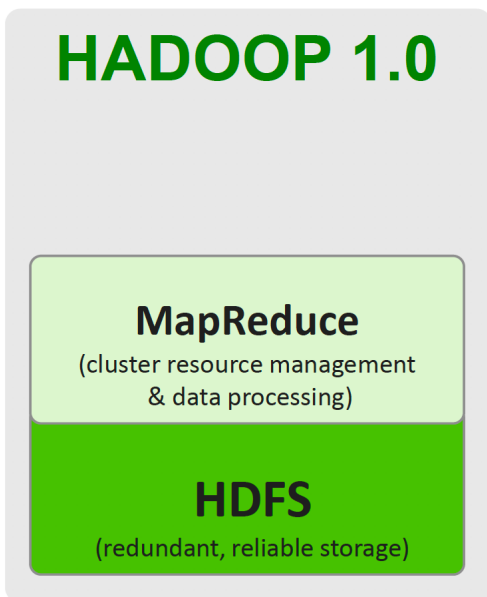


Apache Hadoop 2.0/3.0

Introducing Hadoop 2.0

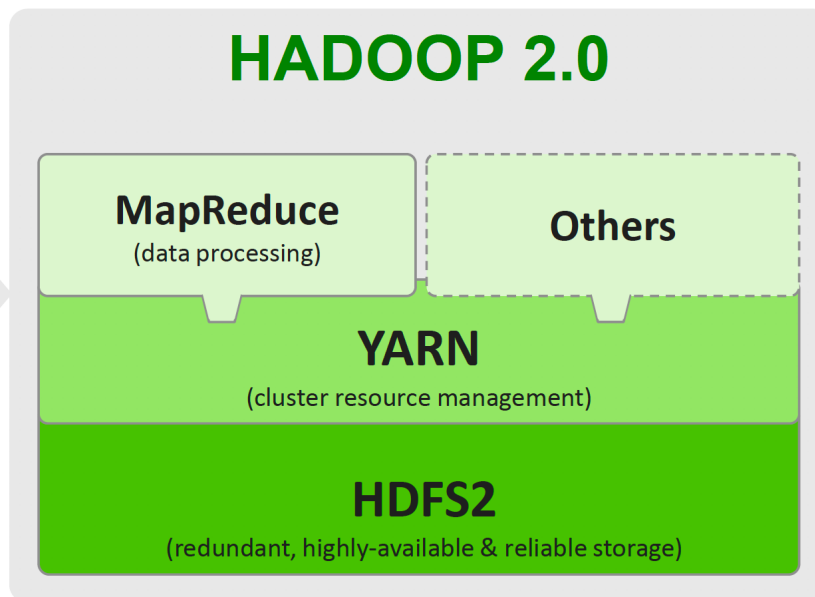
Single Use System

Batch Apps



Multi Purpose Platform

Batch, Interactive, Online, Streaming, ...



© Hortonworks Inc. 2013: "YARN - Hadoop Next Generation Compute Platform", slideshare

Hadoop 2.0 - YARN Architecture

- YARN = “Yet Another Resource Negotiator“

- ResourceManager (RM)

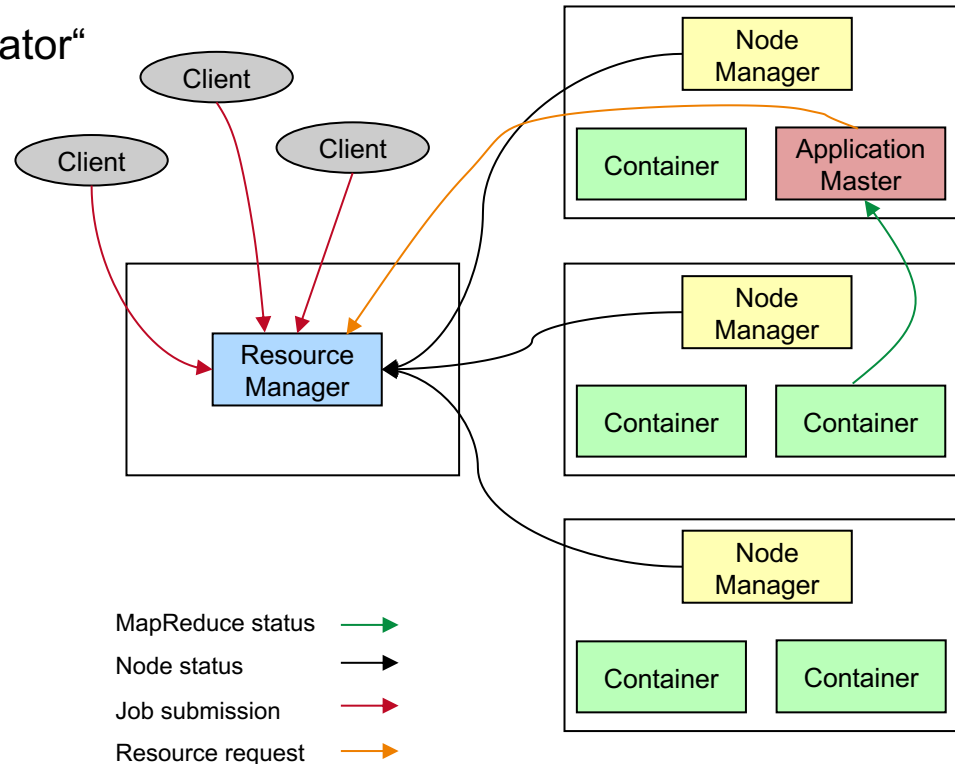
- Central agent - Manages and allocates cluster resources

- NodeManager (NM)

- Per-Node agent - Manages and enforces node resource allocations

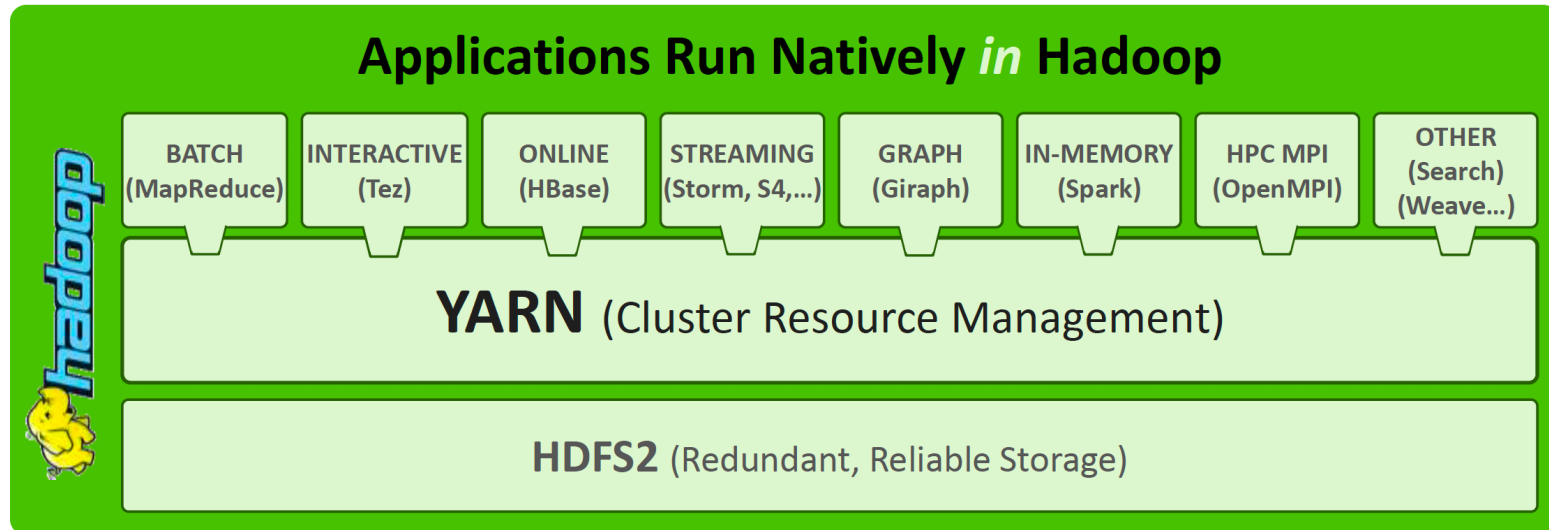
- ApplicationMaster (AM)

- Per-Application - Manages application lifecycle and task scheduling



YARN: Taking Hadoop Beyond Batch

- Store all data in one place...
- Interact with that data in multiple ways
- With predictable performance and quality of service



© Hortonworks Inc. 2013: "YARN - Hadoop Next Generation Compute Platform", slideshare

- V. Vavilapalli, “Apache Hadoop YARN: yet another resource negotiator”, ACM Symposium on Cloud Computing, New York, USA, 2013.